AIML231 — Week 05

Data Preprocessing

Dr Qi Chen

School of Engineering and Computer Science

Victoria University of Wellington

Qi.Chen@vuw.ac.nz

Week Overview

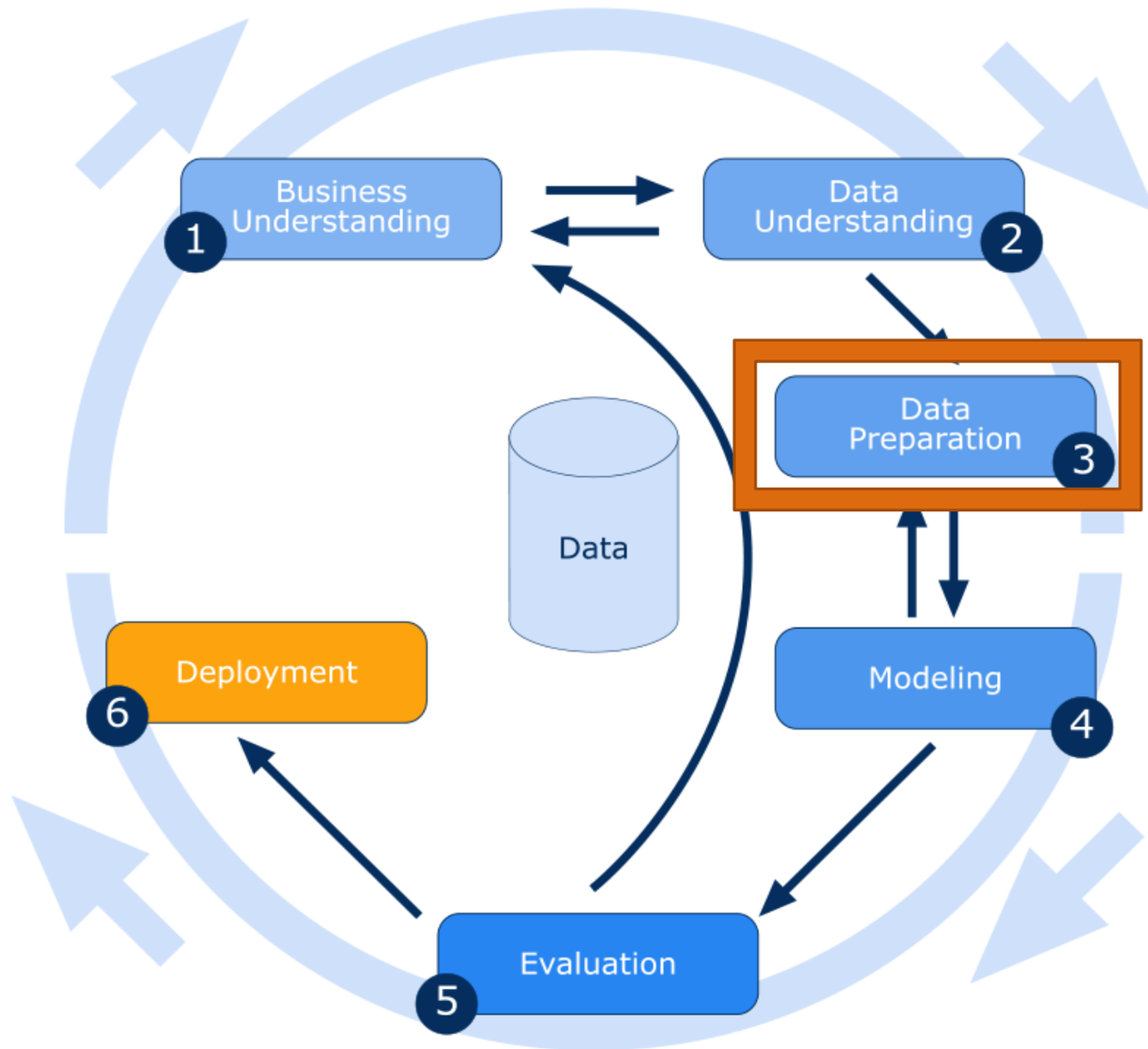
- Introduction of Data Preparation
  - What data preparation includes
  - Why data preparation
  - Avoid data leakage

- Data Preprocessing
  - Categorical Data Encoding
  - Normalisation
  - Discretisation
  - Impute missing values

- Feature Manipulation
  - Dimensionality Reduction
  - Feature Construction
  - Feature Selection

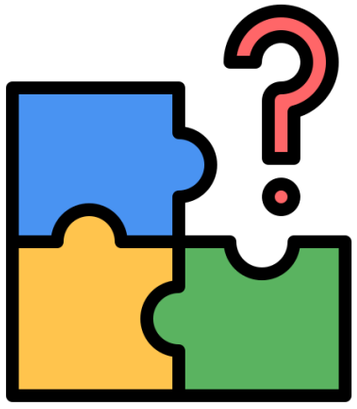
Data Preprocessing/Preparation

- Prepare the final data set(s) for modelling
- Takes over 80% of time and effort in the project



Why Data Preprocessing?

- Data in the real world:
  - incomplete: missing attribute values
  - inconsistent: “03/07/2015”, “March 07, 2015”
  - noisy: containing errors or outliers, gender=“Male”, pregnant = “Yes”
  - large-scale/big data: with a large number of features and instances
  - different types: numeric, nominal, text, Web data, images, audio/video



- Different ML tools use different data formats; Different ML methods have different requirements

- Garbage in, garbage out

Data Leakage

- A Problem with naive data preparation - data leakage
- Information/knowledge about the holdout dataset, e.g. a test dataset, leaks into the data used to train the model
- result in an incorrect estimate of model's perdiction performance

Data Preparation->Data Splitting-> Modelling ✗

Data Splitting-> Data Preparation-> Modelling ✓

Data Preprocessing

- Two main types of features:
  - Numerical features: discrete (integers) vs continuous
  - Categorical features: nominal vs ordinal.

- Categorical Features:
  - *Encoder* as numerical features.
- Numerical Features:
  - *Normalisation/scaling:* all features have similar ranges
  - *Discretisation:* Converts continuous features into discrete bins.

- Impute missing values

Encoding Categorical Features

- Process of converting the categorical values into numerical values.

- But why?
  - Most machine learning models require numerical input (SVMs, KNN).

One-hot Encoding

- Nominal features
- for each unique value in a categorical column, a new column is added

Country
USA
UK
USA
France
USA
UK

dummy features

USA	UK	France
1	0	0
0	1	0
1	0	0
0	0	1
1	0	0
0	1	0

- *sklearn.preprocessing.OneHotEncoder*

One-hot Encoding: Code Example

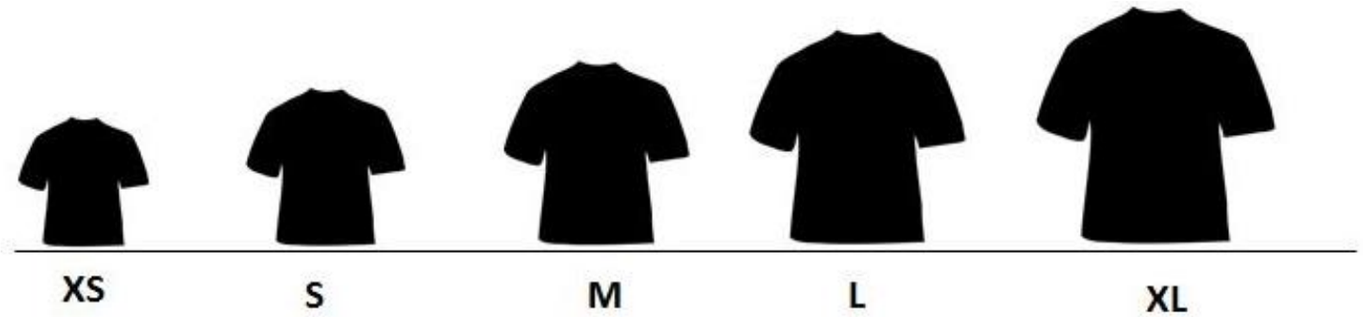
```
from sklearn.preprocessing import OneHotEncoder

# Sample data
data = pd.DataFrame({'Color': ['Red', 'Blue', 'Green']})

# Apply One-Hot Encoding
encoder = OneHotEncoder()
encoded = encoder.fit_transform(data[['Color']])
```

Ordinal Encoding

- Ordinal features that have values with a natural rank order.
  - E.g.



- Ordinal encoding: assign integers to values in certain order.

Original values	Encoding values
XS	0
S	1
M	2
L	3
XL	4

- *sklearn.preprocessing.OrdinalEncoder*

Ordinal Encoding: Code Example

```python
import pandas as pd
from sklearn.preprocessing import OrdinalEncoder

# Sample data
data = pd.DataFrame({'Education': ['High School', 'Bachelor', 'Master', 'PhD']})

# Define order
education_order = [['High School', 'Bachelor', 'Master', 'PhD']]

# Apply Ordinal Encoding
encoder = OrdinalEncoder(categories=education_order)
data['Education_encoded'] = encoder.fit_transform(data[['Education']])
```

Scaling Numerical Features

- Numerical features may have *different* ranges.

- They should be *scaled* to be in the *same* range.

- But why?
  - Features with a larger range will *dominate* the models.
  - Especially in *distance* computations.

- Two scaling methods:
  - Linear (Min-Max) Scaling
  - Z-score Standardisation

Linear (Min-Max) Scaling

- We usually scale so that all samples fall in the range [0, 1].
- For each feature *f*:
  - find the *maximum* (x_{max}) and *minimum* (x_{min}) values
  - for any value x, the scaled value $x' = \frac{x - x_{min}}{x_{max} - x_{min}}$
  - after scaling, the min value is 0, and the max value is 1
  - preserve the linear ratio between feature values after scaling.
- For a specified range, [New_{min}, New_{max}]

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}} \times (New_{max} - New_{min}) + New_{min}$$

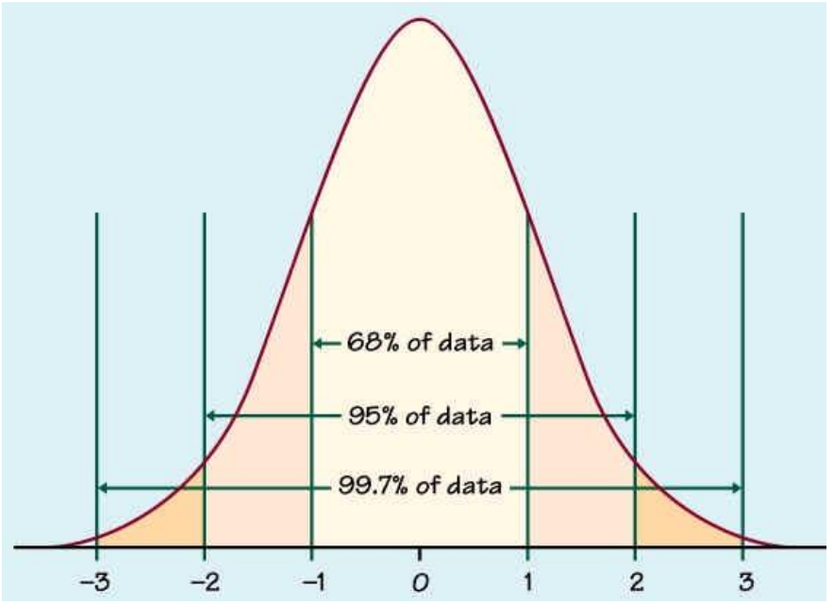
- *sklearn.preprocessing.MinMaxScaler*

Z-score Standardisation

- Centre scaling: scales the *distribution of samples*, so they have a *mean of 0*, and a *standard deviation of 1*.
- For a feature with mean μ and standard deviation σ, the scaled value z of for a raw value x is given by:

$$z = \frac{x - \mu}{\sigma}$$

- With a normally distributed feature, it is easy to map from scaled values to probability



- *sklearn.preprocessing.StandardScaler*

Linear Scaling or Z-score Standardisation?

- If your numerical feature is normally distributed, use z-score standardisation.
- Otherwise, use linear scaling.

- If you have a mixture of normal and non-normal features, you should use linear scaling on all of them—so you can scale them all to the *same range*.

Scaling: Code Example

```python
import panda as pd
from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import StandardScaler

# Sample data
data = pd.DataFrame({'Value': [10.1, 20.2, 30.5, 40.1, 50.2]})

# Apply Min-Max Scaling
scaler = MinMaxScaler()
data['Linear_Scaled'] = scaler.fit_transform(data[['Value']])

# Apply Z-score Standardisation

scaler = StandardScaler()
data['Standardised'] = scaler.fit_transform(data[['Value']])

print(data)
```

Discretisation

- Discretisation: a process of converting numerical features such as price and weight into ordinal features.
- But why?
  - Some algorithms prefer/require categorical inputs, e.g. DT, rule-based algorithms.
  - Visualisation: histograms…

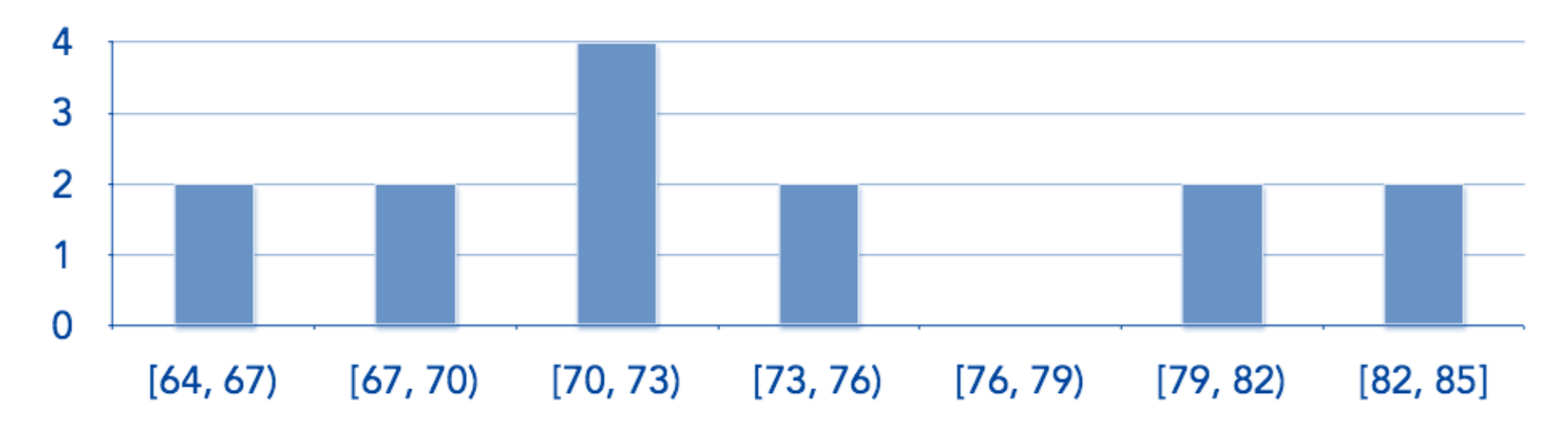
Two approaches:

- Equal-Width/Uniform
- Equal-Depth/Quantile
- *sklearn.preprocessing. KBinsDiscretizer*

Equal-Width/Uniform

- Discretise a numerical attribute to N equal-sized bins :
  - Find the Max and Min values of the attribute.
  - Divides the range [Min, Max] into N intervals of equal size.
  - The width of intervals: W=(Max - Min)/N.

- Example: Temperature values are {85, 80, 83, 70, 64, 65, 68, 71, 69, 72, 75, 75, 81,72} and we want 7 bins.
  - Max = 85, Min = 64, W = (85-64)/7 = 3.

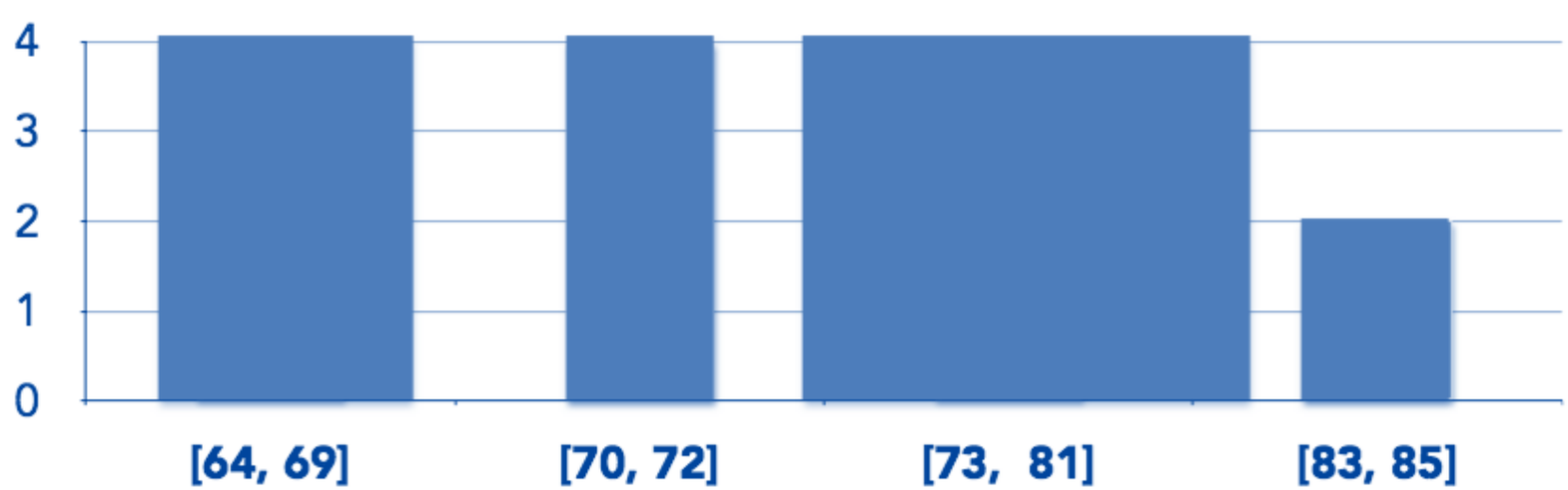


Equal-Depth/Quantile

- Discretise a numerical feature to N bins that each hold (approximately) the same number of samples:
  - Sort feature values in ascending order.
  - Determine the number of samples per bin

    $\lceil number\ of\ samples/N \rceil$

- Example: Temperature values are {85, 80, 83, 70, 64, 65, 68, 71, 69, 72, 75, 75, 81,72} and we want 4 bins.
  - Sort the data: {64, 65, 68, 69, 70, 71, 72, 72, 75, 75, 80, 81, 83, 85}
  - $\lceil 14/4 \rceil$ -> roughly 4 samples per bin



Equal-Width or Equal-Depth

- Equal-Width:

  (+) *Preserve the data scale:* Bin widths are consistent, maintaining uniform intervals across the entire range.

  (-) May result in *uneven data distribution*: Some bins may have few or no samples, especially with skewed data.

  - Good for maintaining *numeric ranges* like temperature, age…

- Equal-Depth:

  (+) *Better for handling skewed distributions:* each bin contains approximately the same number of samples.

  (-) *Bin widths vary*: making interpretation harder.

  - Good for *ranking, percentiles*…

Discretisation: Code Example

```python
import numpy as np
import pandas as pd
from sklearn.preprocessing import KBinsDiscretizer

# Sample data
data = pd.DataFrame({'Value': [3, 7, 11, 13, 16, 18, 21, 24, 27, 30]})

# Equal-Width Discretization
kbins_width = KBinsDiscretizer(n_bins=4, encode='ordinal', strategy='uniform')
data['Equal-Width'] = kbins_width.fit_transform(data[['Value']])

# Equal-Depth (Quantile) Discretization
kbins_depth = KBinsDiscretizer(n_bins=4, encode='ordinal', strategy='quantile')
data['Equal-Depth'] = kbins_depth.fit_transform(data[['Value']])

print(data)
```

Missing Values

- Values for one or more variables are missing from recorded instances.
- Missing data is a common issue in almost every real dataset.

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	?	No
2	? NaN	?	100K	No
3	No	Single	70K	No
4	Yes	Married	?	No
5	No	?	95K	Yes
6	?	Married	60K	No
7	Yes	Divorced	220K	No
8	No	?	?	Yes
9	?	Married	75K	No
10	No	Single	90K	Yes

Handling missing values

- Deletion approaches
  - Removes all records containing missing values.
  - Suitable only when less than 5% of the data is missing to avoid significant data loss.

- Imputation (estimation) approaches
  - Estimates missing values based on available data.
  - Common methods:
    - Mean/Mode Imputation.
    - KNN Imputation.

Delete Imcomplete Data Instances

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	?	Married	?	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	?	Divorced	?	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	?	?	75K	No
10	No	Single	90K	Yes

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
10	No	Single	90K	Yes

Delete Features with Missing Values

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	?	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	?	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	?	Married	75K	No
10	No	Single	90K	Yes

Tid		Marital Status	Taxable Income	Cheat
1		Single	125K	No
2		Married	100K	No
3		Single	70K	No
4		Married	120K	No
5		Divorced	95K	Yes
6		Married	60K	No
7		Divorced	220K	No
8		Single	85K	Yes
9		Married	75K	No
10		Single	90K	Yes

Imputation Approaches

- Mean imputation: for numerical features.
  - Fills with the average complete values.
  - *sklearn.impute.SimpleImputer (strategy='mean')*
- Mode imputation: for categorical features.
  - Fills with the most frequent value.
  - *sklearn.impute.SimpleImputer (strategy='most_frequent')*

- KNN imputation
  - Find *K* nearest neighbours using observed values.
  - Estimate the missing value by the mean/mode from the *K* neighbours.
  - *sklearn.impute.KNNImputer()*

Imputation Approaches

- Mean/Mode imputation: based on *all* available samples.
  - (+) Fast, doesn't change the mean/mode of features.
  - (-) Does not consider relationships between features.

- KNN imputation: based on the *K nearest* samples.
  - (+) Consider relationships between features, so often more accurate.
  - (-) High computational complexity, choice of *K.*